

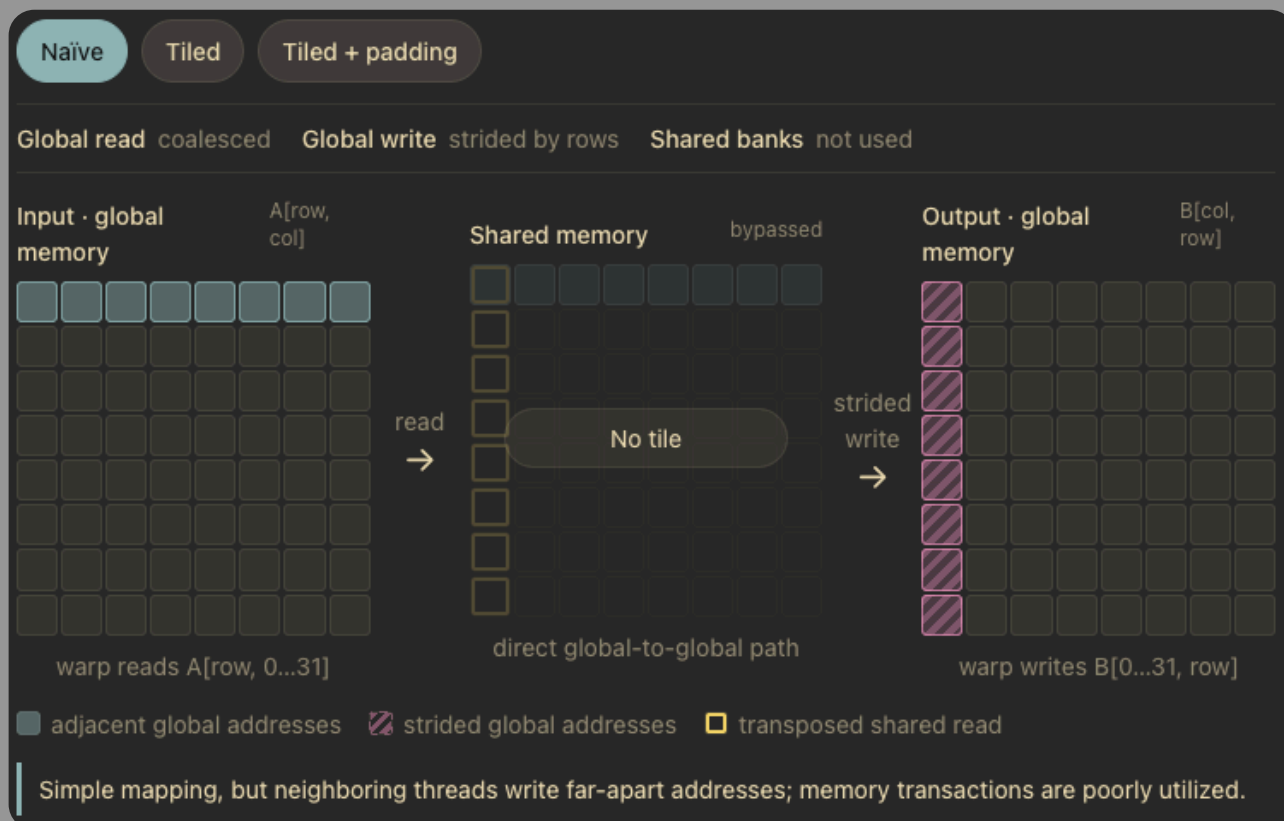
$$A: M \times N \rightarrow B: N \times M$$

$$B[col][row] = A[row][col]$$

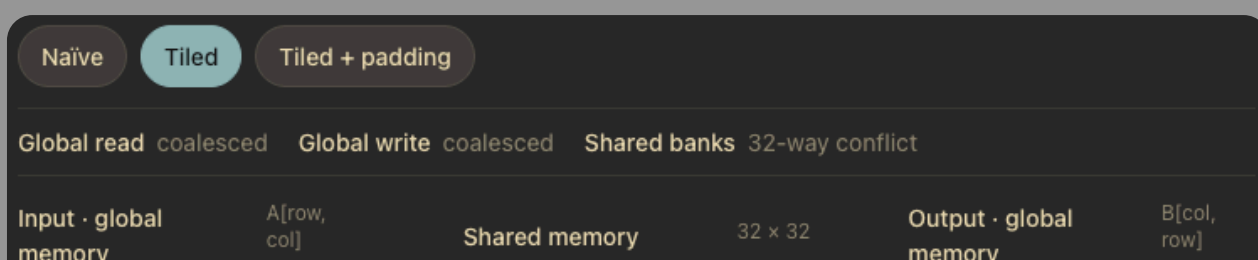
cuda memory is linear. With row-major storage:

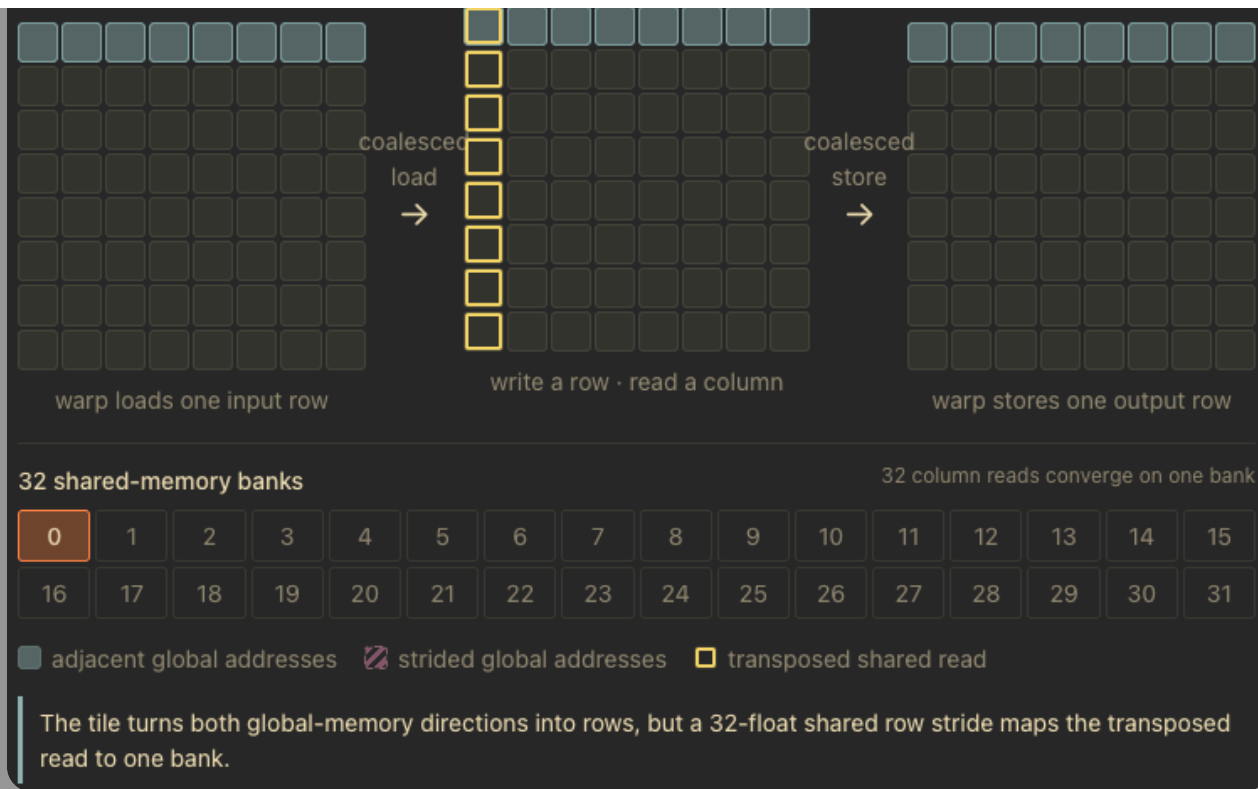
$$A[idx] = row \times N + col$$

$$B[idx] = col \times M + row$$

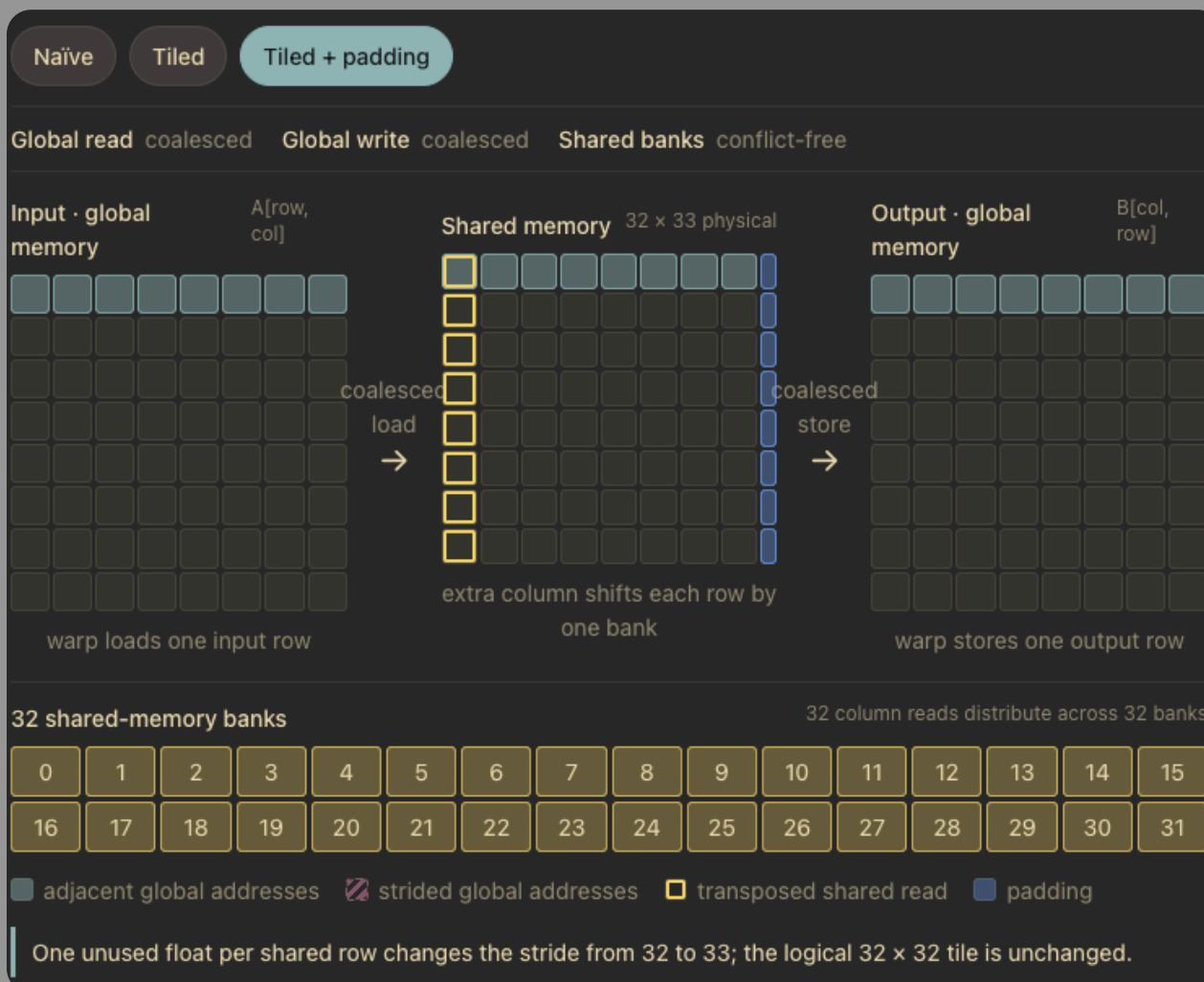


→ each thread only copies one value.
 Reads are coalesced, but writes are strided





→ shmem rotates the access pattern,
making both global reads and writes coalesced



→ the physical file becomes 32x33

the extra col. changes the shmem row stride
from 32 to 33,

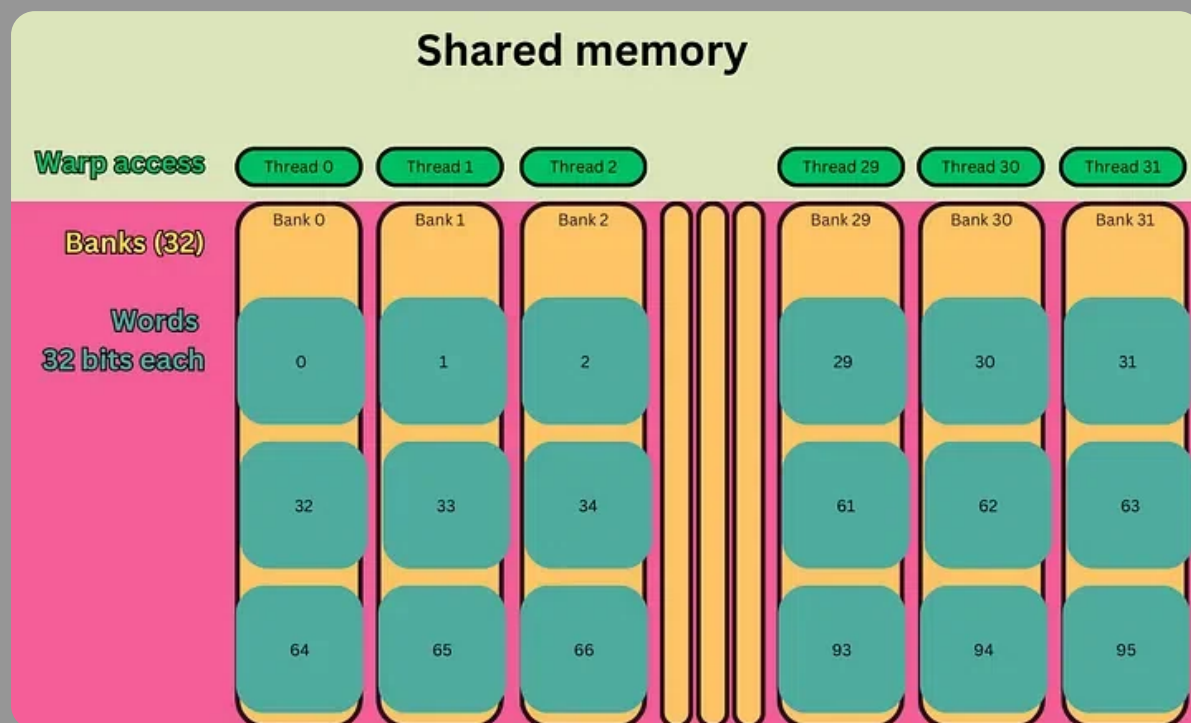
distributing a warp's accesses across all 32 banks

→ padding does not alter the matrix or output.

It only changes address alignment

⇒ **BANK CONFLICT**

+ occurs when multiple threads within the same warp
attempt to access memory locations that belong to the
same memory bank simultaneously



Example:

Warps size = Shared Memory banks count = 8

Data size = [2x8]

If each thread attempts access items by column *in the matrix* (i.e., warp0 => [0, 8]), it will lead to a Bank Conflict.

A matrix layout:

[[0, 1, 2, 3, 4, 5, 6, 7]

[8, 9, 10, 11, 12, 13, 14, 15]]

An address calculation
formula:

$index = x + y * width$

Warp access

Thread 0

Thread 1

Thread 2

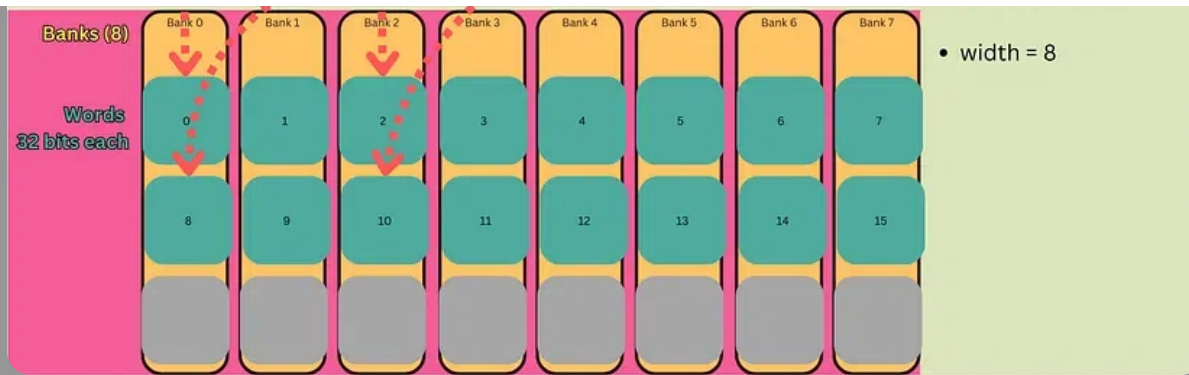
Thread 3

Thread 4

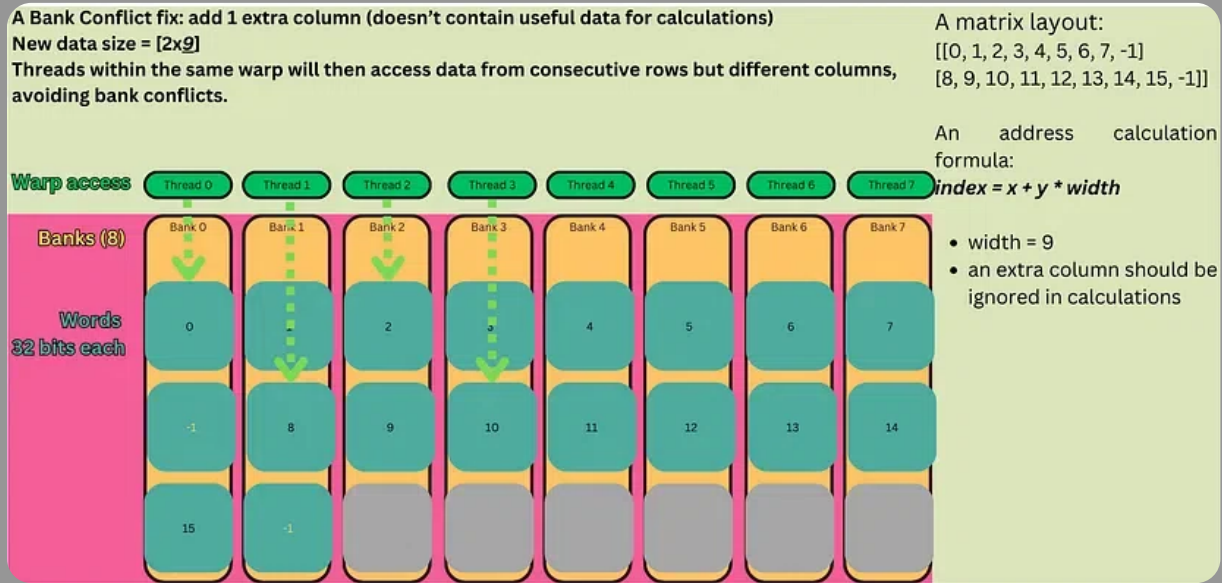
Thread 5

Thread 6

Thread 7



+1 padding



=> padding changes the shmem stride from 32 to 33, preventing col. accesses from repeatedly landing in the same bank

"Why is naive transpose slow?"

=> each thread only copies one value

"Are the reads/writes coalesced?" → unified

=> reads are coalesced
 writes are strided.

"Why does shmem improve the access pattern?"

=> shmem rotates the access pattern making both reads and writes coalesced.

"why does adding one col. padding sometimes improves performance?"

$\Rightarrow$ padding changes the stride; preventing col. accesses from repeatedly loading in the same bank.